CPS721 Assignment 2: Question 1, List Equality

Name	Email	Student ID
Natnael Michael		500982899
Taarush Jain	taarush94@gmail.com	501010905
Vladimirs Kalinins		501291214

Method: for each pair, both lists are rewritten in the same (standard comma-separated) notation, using `[H | [A, B, ...]] = [H, A, B, ...]` and `[A, B | [C, D]] = [A, B, C, D]`. The lists are then matched element by element, from the first element onwards. Two lists can be made identical only if every pair of corresponding elements can be made identical with one consistent set of variable values, and the lists have the same length (or a tail variable absorbs the remaining elements).

a. Can be made identical

List 1: `[truck, T, boat | [T, car, plane]]`
List 2: `[N | [boat, M | [P | Q]]]`

Rewriting both in comma notation:

List 1 = `[truck, T, boat, T, car, plane]` (6 elements)
List 2 = `[N | [boat, M | [P | Q]]] = [N, boat, M | [P | Q]] = [N, boat, M, P | Q]` (4 elements, then the tail `Q`)

Position	List 1	List 2	Result
1	`truck`	`N`	`N = truck`
2	`T`	`boat`	`T = boat`
3	`boat`	`M`	`M = boat`
4	`T` (= `boat`)	`P`	`P = boat`
rest	`[car, plane]`	`Q`	`Q = [car, plane]`

Every position matches, so with `N = truck`, `T = boat`, `M = boat`, `P = boat` and `Q = [car, plane]` both lists become `[truck, boat, boat, boat, car, plane]`.

b. Cannot be made identical

List 1: `[a, [M | [b, c]], d]`
List 2: `[a, [b, [b, c]] | T]`

Rewriting both in comma notation:

List 1 = `[a, [M, b, c], d]`, since `[M | [b, c]] = [M, b, c]`
List 2 = `[a, [b, [b, c]] | T]` (2 elements, then the tail `T`)

Position	List 1	List 2	Result
1	`a`	`a`	match
2	`[M, b, c]`	`[b, [b, c]]`	**fails** (see below)

The second elements are both lists, so they must match element by element too:

- First elements: `M` and `b`, so `M = b`.
- Second elements: `b` and `[b, c]`. The constant `b` is not a list, so it can never equal `[b, c]`. This fails.

The two inner lists also have different lengths (`[M, b, c]` has 3 elements, `[b, [b, c]]` has 2), so they could not match for any value of `M`. Since the second elements cannot be made identical, the lists cannot be made identical, whatever `T` is.

c. Can be made identical

List 1: `[X | [X | [[X | [[X]]]]]]`
List 2: `[b | M]`

Rewriting List 1 from the inside out:

- `[X | [[X]]] = [X, [X]]`, so `[[X | [[X]]]] = [[X, [X]]]` (a list with one element, `[X, [X]]`).
- `[X | [[X, [X]]]] = [X, [X, [X]]]`.
- `[X | [X, [X, [X]]]] = [X, X, [X, [X]]]`.

So List 1 = `[X, X, [X, [X]]]`, which in bar notation is `[X | [X, [X, [X]]]]`. List 2 is `[b | M]`.

Part	List 1	List 2	Result
head	`X`	`b`	`X = b`
tail	`[X, [X, [X]]]`	`M`	`M = [b, [b, [b]]]`

With `X = b` and `M = [b, [b, [b]]]`, both lists become `[b, b, [b, [b]]]`.

d. Cannot be made identical

List 1: `[U | [W | [U]]]`
List 2: `[the, quick, brown, fox, W]`

Rewriting List 1 in comma notation: `[W | [U]] = [W, U]`, so List 1 = `[U, W, U]` (3 elements). List 2 has 5 elements.

Position	List 1	List 2	Result
1	`U`	`the`	`U = the`
2	`W`	`quick`	`W = quick`
3	`U` (= `the`)	`brown`	**fails**: `the` and `brown` are different constants

The match fails at position 3, because `U` already has the value `the` and cannot also be `brown`. Even apart from that, List 1 has 3 elements and List 2 has 5, and neither list ends in a tail variable that could absorb the extra elements (`fox` and `W`). So the lists cannot be made identical.

e. Can be made identical

List 1: `[Fox, [[in], socks], [on], box, on | [[knox]]]`
List 2: `[[The, cat], [[in], The], Hat | [Comes | Back]]`

Rewriting both in comma notation:

List 1 = `[Fox, [[in], socks], [on], box, on, [knox]]` (6 elements)
List 2 = `[[The, cat], [[in], The], Hat, Comes | Back]` (4 elements, then the tail `Back`)

Position	List 1	List 2	Result
1	`Fox`	`[The, cat]`	`Fox = [The, cat]`
2	`[[in], socks]`	`[[in], The]`	`[in] = [in]` matches, and `The = socks`, so `Fox = [socks, cat]`
3	`[on]`	`Hat`	`Hat = [on]`
4	`box`	`Comes`	`Comes = box`
rest	`[on, [knox]]`	`Back`	`Back = [on, [knox]]`

Every position matches, so with `Fox = [socks, cat]`, `The = socks`, `Hat = [on]`, `Comes = box` and `Back = [on, [knox]]` both lists become `[[socks, cat], [[in], socks], [on], box, on, [knox]]`.

All five results were also checked in ECLiPSe 7.1 by unifying each pair with =: pairs a, c and e succeed with the values above, and pairs b and d fail.